

2.1 Abordagem Mobile-First

Essa é uma ótima prática para entender o conceito de **Mobile-First**, onde estilizamos primeiro para telas menores e depois usamos as `@media queries` para adaptar o layout conforme a tela cresce.

Aqui está o código completo e organizado para você testar:

Código HTML e CSS

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício de Responsividade</title>
  <style>
    /* --- Estilos Mobile (Padrão) --- */

    button {
      width: 100%;
      padding: 10px;
      cursor: pointer;
    }

    p {
      text-align: left;
      font-size: 16px;
      margin-top: 20px;
    }

    /* --- Tablet (768px ou mais) --- */
    @media (min-width: 768px) {
      button {
        width: 200px;
      }
    }

    /* --- Desktop (1024px ou mais) --- */
    @media (min-width: 1024px) {
      p {
        text-align: center;
        font-size: 20px;
      }
    }
  </style>
</head>
<body>

  <button>Enviar</button>

  <p>
    Este texto muda de alinhamento e tamanho dependendo da largura da sua tela.
  </p>

</body>
</html>
```

O que cada parte faz:

- **Mobile (Base):** O CSS fora das chaves `@media` é aplicado a todos os dispositivos. Definimos o botão com `width: 100%` e o parágrafo alinhado à esquerda.
- **Breakpoint 768px:** Quando a tela atinge essa largura (comum em tablets), a regra de `200px` sobrescreve os 100% iniciais do botão.
- **Breakpoint 1024px:** Ao chegar em telas de computadores, o parágrafo recebe as novas propriedades de centralização e aumento da fonte.

Dica de ouro: Para testar isso no navegador, clique com o botão direito na página, vá em **Inspecionar** e altere o tamanho da janela ou use o ícone de simulação de dispositivos móveis (o ícone de celular/tablet no menu de inspeção).

2.2 Breakpoints (Pontos de Quebra)

Dando continuidade ao exercício, vamos aplicar agora o conceito de **Flexbox** para o layout das caixas. No CSS moderno, o Flexbox facilita muito essa transição de uma coluna (mobile) para linhas (desktop).

Aqui está o código atualizado com a seção dos cards:

Código HTML e CSS (Parte 2)

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exercício de Responsividade - Parte 2</title>
<style>
  /* --- Estilos do Botão e Texto (Parte 1) --- */
  button { width: 100%; padding: 10px; }
  @media (min-width: 768px) { button { width: 200px; } }

  p { text-align: left; font-size: 16px; margin: 20px 0; }
  @media (min-width: 1024px) {
    p { text-align: center; font-size: 20px; }
  }

  /* --- Estilos dos Cards (Parte 2) --- */

  /* No Mobile: Divs já são block por padrão, então ficam empilhadas */
  .container-cards {
    display: flex;
    flex-direction: column; /* Garante o empilhamento vertical */
    gap: 10px;
  }

  .card {
    background-color: #f0f0f0;
    padding: 20px;
    border: 1px solid #ccc;
    text-align: center;
  }
```

```

/* Tablet (768px): Colocar lado a lado */
@media (min-width: 768px) {
  .container-cards {
    flex-direction: row; /* Alinha os itens horizontalmente */
    justify-content: space-between;
  }
  .card {
    flex: 1; /* Faz as caixas terem tamanhos iguais */
  }
}

/* Desktop (1024px): Aumentar margem e borda */
@media (min-width: 1024px) {
  .container-cards {
    gap: 40px; /* Aumenta a distância entre as caixas */
  }
  .card {
    border-width: 5px; /* Deixa a borda mais grossa */
    border-color: #333;
  }
}
</style>
</head>
<body>

<button>Enviar</button>
<p>Texto responsivo abaixo do botão.</p>

<div class="container-cards">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
</div>

</body>
</html>

```

Explicação das novidades:

- **display: flex e flex-direction:** No mobile, usamos `column` para que fiquem um abaixo do outro. No breakpoint de 768px, mudamos para `row` para que fiquem lado a lado.
- **flex: 1:** Colocamos isso nos `.card` dentro da media query de 768px para que eles ocupem o mesmo espaço disponível, dividindo a largura da tela igualmente.
- **gap:** É a forma mais moderna e limpa de criar espaçamento entre itens Flexbox. No desktop (1024px), aumentamos esse valor de 10px para 40px.
- **border-width:** Conforme pedido, aumentamos a espessura da borda apenas na tela maior.

Dica: Note que usei um "container" em volta das divs. Isso é uma boa prática para controlar o comportamento do grupo de elementos sem afetar o restante da página.
